

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Ingegneria del Software: Tecniche Avanzate

Progetto ISTA – Evoluzione di CodeSmile Post-Modification Testing

Professore:

**Ch.mo Prof.
Andrea De Lucia**

Tutor di riferimento:

**Giammaria Giordano
Gilberto Recupito**

Studenti:

**Antonio D'Auria
Mat. NF22500063
Emanuele D'Auria
Mat. 0522501918
Luca Casillo
Mat. NF22500037**

ANNO ACCADEMICO 2025/2026

INDICE

Indice

Elenco delle tabelle

1	Introduzione	1
1.1	Obiettivi	1
1.2	Perimetro di Verifica	2
2	Ambiente e Metodo di Esecuzione	2
2.1	Procedura	2
2.2	Classificazione degli Esiti	4
3	Verifica delle Change Request	4
3.1	CR1 – Generazione del Call Graph	4
3.2	CR2 – Estensione della CLI con Opzioni Avanzate	5
3.3	CR3 – Portabilità tra Ambiente Locale e Docker	5
3.4	CR4 – Visualizzazione del Call Graph nella WebApp	6
3.5	Verifica di FPIS e DIS	7
4	Risultati per Componente e Livello di Test	7
4.1	Static Analysis Tool	7
4.1.1	Unit Testing	7
4.1.2	Integration Testing	7
4.1.3	System Testing	8
4.2	Web Application	8
4.2.1	Unit Testing	8
4.2.2	Integration Testing	8
4.2.3	End-to-End Testing	8
4.2.4	Build e Portabilità	9
4.3	Sintesi delle Esecuzioni	9
4.4	Evoluzione della Test Suite	10
5	Riesame degli Incidenti Pre-Modifica	10
5.1	Static Analysis Tool	11
5.1.1	Unit Test Incidents	11
5.1.2	Integration Test Incidents	11
5.1.3	System Test Incidents	11
5.2	Web Application	11
5.2.1	Unit Test Incidents	11
5.2.2	Integration Test Incidents	11
5.2.3	End-to-End Test Incidents	12
6	Valutazione Complessiva	12
6.1	Branch Coverage	12
6.2	Osservazioni Non Bloccanti	12
6.3	Applicazione dei Criteri di Accettazione	13
6.4	Conclusioni	13

ELENCO DELLE TABELLE

2.1 Ambiente di esecuzione post-modifica	2
2.2 Livelli e strumenti impiegati	3
3.1 Verifica dei deliverable di CR1	4
3.2 Verifica dei deliverable di CR2	5
3.3 Verifica dei deliverable di CR3	6
3.4 Verifica dei deliverable di CR4	6
3.5 Copertura dei False Positive e Discovered Impact Set	7
4.1 Confronto dei system test CLI con la baseline	8
4.2 Esiti End-to-End post-modifica	9
4.3 Risultati complessivi del testing post-modifica	9
4.4 Evoluzione dei file di test rispetto alla baseline pre-modifica	10
5.1 Riesame degli incidenti unitari dello Static Analysis Tool	11
5.2 Riesame degli incidenti di integrazione della Web Application	11
5.3 Riesame degli incidenti End-to-End della Web Application	12
6.1 Confronto della branch coverage	12
6.2 Avvisi osservati durante il testing	13
6.3 Valutazione rispetto al Master Test Plan	13

1 Introduzione

Il presente documento riporta le attività di testing eseguite sul sistema CodeSmile dopo l'implementazione delle quattro Change Request. L'obiettivo è verificare con evidenze ripetibili sia la conformità dei nuovi deliverable sia la preservazione dei comportamenti già osservati nello stato pre-modifica.

La descrizione delle modifiche rimane affidata al documento di *Change Requests*; la strategia, i livelli di test e i criteri decisionali sono quelli stabiliti nel *Master Test Plan*. Il presente elaborato si concentra pertanto su esecuzione, risultati e valutazione finale.

1.1 Obiettivi

Le attività post-modifica perseguono i seguenti obiettivi:

- verificare che ogni Change Request soddisfi i deliverable dichiarati;
- esercitare le unità appartenenti agli insiemi di impatto effettivo e i punti di propagazione individuati nell'Impact Analysis;
- sottoporre a regressione i componenti presenti nei False Positive Impact Set, poiché dipendenti dalle unità modificate pur non avendo richiesto interventi;
- controllare in modo specifico le unità appartenenti ai Discovered Impact Set di CR3 e CR4;
- confrontare la branch coverage con i valori pre-modifica;
- applicare i criteri di accettazione definiti nel Master Test Plan.

1.2 Perimetro di Verifica

Il perimetro comprende lo Static Analysis Tool, la CLI, il flusso statico dei servizi backend, la generazione dei report e la Web Application. Sono incluse le modalità di avvio locale e Docker interessate da CR3, nonché il flusso end-to-end che dalla selezione di un progetto conduce alla visualizzazione del Call Graph introdotta da CR4.

La semantica dell’analisi AI-Based, i modelli utilizzati e la qualità delle relative predizioni restano fuori scope, in continuità con il Master Test Plan. I test che richiedono il servizio AI esterno sono quindi classificati come non applicabili e non concorrono all’esito delle Change Request. Tale esclusione non modifica il vincolo di non introdurre alterazioni intenzionali nei componenti preesistenti estranei alle modifiche.

2 Ambiente e Metodo di Esecuzione

Le verifiche sono state eseguite il 4 agosto 2026 in ambiente Windows locale e, per gli aspetti di portabilità, mediante container Linux gestiti da Docker. Le versioni principali sono riportate nella Tabella 2.1.

Tabella 2.1. Ambiente di esecuzione post-modifica

Elemento	Versione o configurazione
Sistema host	Microsoft Windows; servizi locali su interfaccia loopback.
Python	3.11.0, virtual environment del progetto.
Node.js / npm	Node.js 22.13.1; npm 10.9.2.
Frontend	Next.js 14.2.23; Jest; React Testing Library.
End-to-End	Cypress 13.17.0, esecuzione headless con Electron 118.
Container	Docker Engine 28.5.1; rete e servizi definiti da <code>docker-compose.yml</code> .

2.1 Procedura

L’esecuzione ha seguito l’ordine progressivo definito nel Master Test Plan: unità, integrazione, sistema, frontend, flussi end-to-end e smoke test infrastrutturali. Prima di valutare un esito sono state predisposte le sole dipendenze necessarie al livello considerato; un servizio non avviato non è stato interpretato come fault del prodotto.

Tabella 2.2. Livelli e strumenti impiegati

Livello	Strumento	Oggetto della verifica
Unit	PyTest, Jest	Moduli AST, builder, CLI, orchestrazione e componenti UI.
Integration	PyTest, client HTTP	Collaborazione del core, contratti tra gateway e servizi, configurazione degli endpoint.
System	Script CLI automatizzato	Ventuno scenari del corpus pre-modifica e relativi output.
End-to-End	Cypress	Navigazione, upload statico, report e Call Graph dal punto di vista dell'utente.
Build e smoke	Next.js, Docker Compose, HTTP	Compilazione frontend, costruzione delle immagini, avvio e raggiungibilità dei servizi.

Per lo Static Analysis Tool la branch coverage è stata misurata separatamente sulle suite unit e integration tramite `coverage.py`. I test di sistema, Jest e Cypress non sono stati inclusi nella metrica, in conformità alle limitazioni e alle scelte già definite nel Master Test Plan.

2.2 Classificazione degli Esiti

Sono state utilizzate le seguenti classificazioni:

- **PASS**: comportamento osservato conforme a quello atteso;
- **PASS_NO_OUTPUT**: esecuzione corretta senza smell prodotti, coerente con la natura del caso di sistema;
- **SKIP_NO_PY**: caso non eseguibile perché l’input non contiene file Python, già classificato allo stesso modo nella baseline;
- **N/A**: scenario esterno al perimetro, riferito al servizio AI-Based;
- **FAIL**: divergenza riproducibile dal comportamento atteso.

Gli output strutturati dei test di sistema sono stati raccolti in `output/system_testing_post`; la sintesi consente il confronto diretto con gli esiti pre-modifica caso per caso.

3 Verifica delle Change Request

La verifica delle Change Request combina test automatici, controlli manuali mirati e osservazione dei contratti scambiati tra le componenti. Gli esiti riportati in questa sezione sono riferiti ai deliverable definiti nel documento di Change Requests.

3.1 CR1 – Generazione del Call Graph

I test unitari dedicati esercitano l’estrazione AST di moduli, funzioni e metodi, la risoluzione delle chiamate, la costruzione di nodi e archi e la serializzazione dell’artefatto. I test di integrazione verificano la collaborazione con Inspector e ProjectAnalyzer, sia per l’analisi di un singolo progetto sia per l’elaborazione multipla.

Tabella 3.1. Verifica dei deliverable di CR1

Deliverable	Evidenza	Esito
Modulo AST dedicato	Suite di <code>call_graph_extractor</code> e casi su funzioni, metodi e chiamate qualificate.	PASS
Artefatto strutturato	Builder JSON con campi di nodo, archi e informazioni sugli smell; controllo di serializzazione.	PASS
Integrazione nel flusso statico	Test della pipeline Inspector–ProjectAnalyzer e servizio statico; modalità con e senza grafo.	PASS
Analisi singola e multipla	Test di integrazione della CLI e dell’orchestratore su entrambe le modalità.	PASS

Le coperture dei due moduli introdotti raggiungono il 99% per `call_graph_extractor.py` e il 97% per `call_graph_builder.py` nella suite unitaria. Il formato prodotto contiene inoltre gli attributi `smells`, `is_smelly` e `calls_smelly`, necessari alla successiva visualizzazione senza alterare il report testuale preesistente.

3.2 CR2 – Estensione della CLI con Opzioni Avanzate

La CLI espone i quattro parametri richiesti, ne valida le combinazioni e li propaga a ProjectAnalyzer. Oltre ai test automatici, è stata eseguita un’analisi manuale su un progetto contenente un percorso da escludere.

Tabella 3.2. Verifica dei deliverable di CR2

Opzione o deliverable	Comportamento osservato	Esito
-enable-callgraph	Attiva la costruzione del grafo lasciando invariato il comportamento predefinito quando assente.	PASS
-callgraph-output	Salva il JSON nel percorso richiesto; senza -enable-callgraph la combinazione viene respinta.	PASS
-exclude-paths	Il file collocato nella directory esclusa non viene analizzato e non compare tra i nodi del grafo.	PASS
-format {csv,json}	Il report viene generato nel formato selezionato; CSV rimane il default retrocompatibile.	PASS
Documentazione CLI	Il README descrive i quattro flag e include un esempio completo di utilizzo.	PASS

Nel controllo manuale il Call Graph contiene tre nodi e due archi relativi al solo file ammesso; la funzione presente nel percorso escluso non compare nell’artefatto. Il test negativo termina con codice diverso da zero e segnala esplicitamente la dipendenza tra le due opzioni del grafo.

3.3 CR3 – Portabilità tra Ambiente Locale e Docker

La portabilità è stata verificata prima in locale, controllando sia i default degli URL dei servizi sia gli override tramite variabili d’ambiente. La stessa configurazione è stata quindi validata con Docker Compose, costruendo e avviando gateway, servizio di analisi statica e servizio di report.

Tabella 3.3. Verifica dei deliverable di CR3

Aspetto	Evidenza	Esito
Esecuzione locale	Test di configurazione del gateway e 5 test di integrazione HTTP in scope sui servizi locali.	PASS
Configurazione uniforme	Default locali e override d’ambiente risolvono gli endpoint senza modifiche al codice.	PASS
Composizione Docker	docker compose config valida la configurazione; le tre immagini backend vengono costruite correttamente.	PASS
Smoke test dei container	I tre servizi risultano avviati; analisi statica con Call Graph e generazione del report rispondono con HTTP 200.	PASS
Documentazione di avvio	Il README della Web Application distingue i comandi locali e Docker e riporta le variabili configurabili.	PASS

L’analisi statica inviata attraverso il gateway ha restituito una risposta di successo e un grafo valido; il flusso di report ha prodotto l’output richiesto. Il servizio AI-Based non è stato avviato né valutato semanticamente, in quanto esterno al perimetro delle Change Request.

3.4 CR4 – Visualizzazione del Call Graph nella WebApp

La visualizzazione è stata verificata a tre livelli: test Jest del componente, test della pagina di upload e test Cypress sul flusso completo. La risposta statica viene conservata nel contesto del progetto e mostrata insieme ai risultati testuali già presenti.

Tabella 3.4. Verifica dei deliverable di CR4

Deliverable	Comportamento osservato	Esito
Visualizzazione strutturale	Il componente React Flow rappresenta nodi e archi della risposta graph_data.	PASS
Colorazione intelligente	Rosso per nodi smelly, arancione per nodi che chiamano funzioni smelly e verde per nodi clean.	PASS
Zoom, pan e minimappa	I controlli grafici e la minimappa sono presenti nel viewer e disponibili nel flusso end-to-end.	PASS
Selezione interattiva	Il nodo selezionato e gli archi collegati cambiano stile; la sidebar espone identificativo, file e smell.	PASS
Filtri dinamici	Le checkbox mostrano o nascondono le tre categorie e rimuovono gli archi con estremi non visibili.	PASS
Esportazione	I comandi Export JSON ed Export PNG producono i rispettivi artefatti.	PASS
Compatibilità del risultato	Gli smell testuali restano visibili insieme al Call Graph.	PASS

I quattro test Jest dedicati al viewer coprono colorazione, filtri, selezione, sidebar ed esportazione. Lo scenario Cypress verifica gli stessi comportamenti nel browser con il

backend statico attivo. Un ulteriore smoke test interattivo ha confermato il rendering della home page e la navigazione del pulsante *Analyze Project*; la suite end-to-end verifica inoltre tutti e quattro i pulsanti principali della home.

3.5 Verifica di FPIS e DIS

Gli scostamenti individuati nell’Impact Analysis sono stati trattati senza estendere artificialmente i deliverable delle CR. La Tabella 3.5 riporta le evidenze di regressione rilevanti.

Tabella 3.5. Copertura dei False Positive e Discovered Impact Set

CR	Insieme	Verifica	Esito
CR1	FPIS	CLI, GUI e FileUtils esercitati dalle suite unitarie, di integrazione e di sistema; modalità senza grafo preservata.	PASS
CR2	FPIS	Inspector e generazione dei report sottoposti a regressione senza variazioni del contratto precedente.	PASS
CR3	FPIS	I Dockerfile non modificati sono stati usati per costruire le tre immagini backend.	PASS
CR3	DIS	Il nuovo <code>.dockerignore</code> è stato esercitato durante il build dei context e lo smoke test dei container.	PASS
CR4	FPIS	Extractor, builder e ProjectContext verificati dalle suite dedicate e dal flusso completo.	PASS
CR4	DIS	FileInput conserva i percorsi relativi nella selezione multipla; comportamento verificato con Jest e Cypress.	PASS

4 Risultati per Componente e Livello di Test

I risultati post-modifica sono organizzati secondo la stessa separazione adottata nei documenti di testing pre-modifica: Static Analysis Tool e Web Application. All’interno di ciascuna componente gli esiti sono ulteriormente distinti per livello di test, così da rendere immediato il confronto con la baseline senza alterare i conteggi già consolidati.

4.1 Static Analysis Tool

4.1.1 *Unit Testing.* La suite unitaria dello Static Analysis Tool termina con **175 test positivi su 175**. Oltre alla regressione dei moduli preesistenti, sono esercitati l’estrazione AST, il builder del Call Graph, le nuove opzioni CLI e la propagazione dei parametri attraverso Inspector e ProjectAnalyzer.

La branch coverage complessiva raggiunge l’**87%**, superando la baseline pre-modifica dell’**85%**. I moduli `call_graph_extractor.py` e `call_graph_builder.py` raggiungono rispettivamente il 99% e il 97%; la suite rileva inoltre il 97% su `cli_runner.py`, l’83% su `inspector.py` e l’89% su `project_analyzer.py`.

4.1.2 *Integration Testing.* I test di integrazione del core terminano con **9 esiti positivi su 9**. Le pipeline verificate comprendono CLI–ProjectAnalyzer, ProjectAnalyzer–Inspector, Inspector–RuleChecker, il flusso GUI e il contratto Call Graph del servizio statico.

La branch coverage di integrazione è pari al **51%**, rispetto al 34% pre-modifica. L’incremento è coerente con l’introduzione delle pipeline di CR1 e con il trasporto del grafo richiesto da CR4.

4.1.3 System Testing. Il corpus di ventuno casi CLI è stato rieseguito senza modificarne input o criteri di classificazione. Il confronto automatico non mostra differenze di stato, modalità o numero totale di smell.

Tabella 4.1. Confronto dei system test CLI con la baseline

Classificazione	Pre-modifica	Post-modifica	Scostamento
PASS	14	14	0
PASS_NO_OUTPUT	3	3	0
SKIP_NO_PY	4	4	0
FAIL	0	0	0

I casi TC6, TC8 e TC18 continuano a terminare correttamente senza produrre smell. TC1, TC5, TC15 e TC21 conservano la classificazione SKIP_NO_PY, coerente con la preconditione dei rispettivi input. La stabilità puntuale del corpus conferma che l’attivazione opzionale del Call Graph non ha modificato il comportamento predefinito della CLI.

4.2 Web Application

4.2.1 Unit Testing. Le suite Jest del frontend terminano con **32 test positivi su 32**, distribuiti in sette suite. La baseline comprendeva sei suite e ventisei test; l’incremento corrisponde a una nuova suite e sei casi aggiuntivi. Quattro test sono dedicati al Call Graph viewer e verificano colorazione, filtri, selezione, sidebar ed esportazione. I test della pagina di upload controllano inoltre la conservazione dei percorsi relativi e la presenza congiunta di grafo e risultati testuali.

4.2.2 Integration Testing. Dei sette test di integrazione raccolti, i cinque appartenenti al perimetro statico e di reporting terminano con esito positivo. Sono inclusi i default e gli override di configurazione del gateway, i due contratti di analisi statica e il flusso di generazione del report. I due test riferiti al servizio AI esterno sono classificati **N/A** e non concorrono all’esito.

4.2.3 End-to-End Testing. Le quattro specifiche Cypress registrano ventiquattro scenari: i ventidue in scope terminano positivamente e i due casi AI-Based rimangono esplicitamente pending. La distribuzione degli esiti è riportata nella Tabella 4.2.

Tabella 4.2. Esiti End-to-End post-modifica

Specifica	PASS in scope	N/A
homepage.spec.cy.ts	5	0
report.spec.cy.ts	5	0
upload-project.spec.cy.ts	6	1
upload-python.spec.cy.ts	6	1
Totale	22	2

Gli scenari verificano la navigazione dei quattro pulsanti della home, il reporting, l’upload statico e tutte le interazioni richieste per CR4. Un ulteriore smoke test interattivo ha confermato il rendering della home page e la navigazione verso *Analyze Project*.

4.2.4 Build e Portabilità. La build di produzione Next.js termina correttamente, includendo controllo TypeScript, lint e generazione statica di nove pagine. Per CR3 sono stati verificati i servizi sia in locale sia tramite Docker Compose: la configurazione è valida, le tre immagini backend vengono costruite, gateway, servizio statico e servizio di report risultano avviati e i flussi statico e report rispondono con HTTP 200. I default locali e gli override tramite variabili d’ambiente non richiedono modifiche al codice.

4.3 Sintesi delle Esecuzioni

Tabella 4.3. Risultati complessivi del testing post-modifica

Suite o controllo	Risultato	Note	Esito
Unit test Python	175 / 175	Core statico, CLI, Call Graph, GUI e report.	PASS
Integration test Python	9 / 9	Pipeline core, CLI, GUI e contratto Call Graph del servizio statico.	PASS
System test CLI	21 casi	14 PASS, 3 PASS_NO_OUTPUT, 4 SKIP_NO_PY; nessuna variazione rispetto alla baseline.	PASS
Web integration	5 / 5 in scope	Due test AI raccolti ma esclusi come N/A.	PASS
Frontend Jest	32 / 32	7 suite, incluse 4 verifiche dedicate al Call Graph viewer.	PASS
Frontend build	1 / 1	TypeScript, lint e generazione statica di 9 pagine.	PASS
Cypress End-to-End	22 / 22 in scope	24 scenari registrati; 2 scenari AI pending e classificati N/A.	PASS
Docker e smoke HTTP	Configurazione valida	3 immagini e 3 servizi backend; flussi statico e report con HTTP 200.	PASS

Nel complesso non sono stati osservati fallimenti riproducibili nelle componenti in scope. Le esclusioni non derivano da test falliti: riguardano esclusivamente gli scenari AI-Based già esterni alla strategia di validazione.

4.4 Evoluzione della Test Suite

La classificazione è stata ottenuta confrontando a livello di file la baseline consolidata al termine della fase pre-modifica con lo stato sottoposto a testing post-modifica. Sono inclusi i file di test automatico e il supporto condiviso delle fixture; un file modificato può contenere sia adeguamenti di casi esistenti sia nuovi casi aggiunti nello stesso modulo.

Tabella 4.4. Evoluzione dei file di test rispetto alla baseline pre-modifica

Categoria	N.	File reali rilevati dal confronto
Aggiunti	7	test/conftest.py; test/integration_testing/test_callgraph_pipeline.py; test/integration_testing/test_static_service_callgraph.py; test/unit_testing/call_graph/test_call_graph_builder.py; test/unit_testing/call_graph/test_call_graph_extractor.py; webapp/_tests_/CallGraphViewer.test.tsx; webapp/integration_tests/test_gateway_configuration.py.
Modificati	16	test/integration_testing/: test_analyzer_to_inspector.py, test_cli_to_analyzer.py, test_full_integration_with_cli.py, test_full_integration_with_gui.py, test_gui_to_analyzer.py; test/unit_testing/: cli/test_cli_runner.py, components/test_inspector.py, components/test_project_analyzer.py, gui/test_code_smell_detector_gui.py; webapp/_tests_/upload-project/: ProjectComponent.test.tsx, page.test.tsx; tutti i quattro file in webapp/cypress/e2e/; webapp/integration_tests/test_gateway_to_inspector.py.
Eliminati	0	Nessun file di test è stato eliminato rispetto alla baseline.
Invariati	30	Tutti i 16 file in test/unit_testing/detection_rules/; i 4 file in test/unit_testing/code_extractor/; test/integration_testing/test_inspector_to_rulechecker.py; test/unit_testing/cli/test_file_utils.py; test/unit_testing/components/test_rule_checker.py; test/unit_testing/report/test_report_generator.py; webapp/_tests_/report/ChartSection.test.tsx; webapp/_tests_/report/page.test.tsx; webapp/_tests_/upload-project/ProjectContext.test.tsx; webapp/_tests_/upload-python/page.test.tsx; webapp/integration_tests/test_gateway_to_generate_report.py; webapp/integration_tests/test_gateway_to_model.py.

La baseline comprendeva 46 file nel perimetro considerato; lo stato post-modifica ne comprende 53. La differenza coincide con i sette file aggiunti e il confronto non rileva rinominazioni o eliminazioni.

5 Riesame degli Incidenti Pre-Modifica

Gli identificativi introdotti nel *Pre-Modification Test Incident Report* sono mantenuti nella valutazione post-modifica. La struttura riprende la separazione per componente e livello di test del documento originale, così da distinguere gli incidenti già risolti e sottoposti a regressione da quelli legati al servizio AI esterno.

5.1 Static Analysis Tool

5.1.1 *Unit Test Incidents.* Entrambi gli incidenti unitari restano chiusi e non si sono ripresentati nei 175 test post-modifica.

Tabella 5.1. Riesame degli incidenti unitari dello Static Analysis Tool

ID	Stato nel report	Verifica post-modifica	Esito
INC-SAU-01	Risolto mediante allineamento del test	test_analyze_project_empty_directory termina positivamente nella suite unitaria completa.	PASS
INC-SAU-02	Risolto eliminando la flakiness Tkinter	test_run_program_missing_paths termina senza TclError nella suite GUI.	PASS

5.1.2 *Integration Test Incidents.* Il report pre-modifica non registrava incidenti nei test di integrazione dello Static Analysis Tool. La suite estesa termina con 9 test positivi su 9 e non introduce nuovi incidenti a questo livello.

5.1.3 *System Test Incidents.* Il report pre-modifica distingueva gli esiti SKIP_NO_PY e PASS_NO_OUTPUT senza classificarli come fault di detection. La riesecuzione mantiene esattamente gli stessi casi e le stesse classificazioni; non sono quindi emersi incidenti di sistema post-modifica.

5.2 Web Application

5.2.1 *Unit Test Incidents.* Nel report pre-modifica non erano presenti incidenti unitari della Web Application. Le sette suite Jest e i 32 casi post-modifica terminano positivamente, senza nuovi incidenti a questo livello.

5.2.2 *Integration Test Incidents.* La Tabella 5.2 riesamina i cinque identificativi di integrazione mantenendo gli stati e il perimetro definiti nel report pre-modifica.

Tabella 5.2. Riesame degli incidenti di integrazione della Web Application

ID	Stato nel report	Verifica post-modifica	Esito
INC-WAU-01	Risolto nel test	Contratto statico senza smell verificato con il campo success; test positivo.	PASS
INC-WAU-02	Risolto nel test	Contratto statico con smell verificato con il campo success; test positivo.	PASS
INC-WAU-03	Risolto nel test	Il report conserva filename come stringa; test gateway-report positivo.	PASS
INC-WAU-04	Non risolto; dipendenza AI esterna	Test escluso dal perimetro in coerenza con il Master Test Plan.	N/A
INC-WAU-05	Non risolto; dipendenza AI esterna	Test escluso dal perimetro in coerenza con il Master Test Plan.	N/A

I tre incidenti di contratto risolti restano chiusi. I due ID associati al servizio AI non sono riaperti né considerati fallimenti: rimangono non applicabili al perimetro delle Change Request.

5.2.3 *End-to-End Test Incidents*. La Tabella 5.3 riporta gli esiti dei due incidenti End-to-End già registrati nel report pre-modifica.

Tabella 5.3. Riesame degli incidenti End-to-End della Web Application

ID	Stato nel report	Verifica post-modifica	Esito
INC-WAE-01	Risolto eliminando la race condition	Lo scenario should handle API failure gracefully termina positivamente nella suite Cypress.	PASS
INC-WAE-02	Non risolto; modalità AI-Based	Lo scenario rimane escluso esplicitamente dalla suite in scope.	N/A

Il riesame complessivo non evidenzia riaperture degli incidenti risolti. Gli unici tre ID senza esito PASS coincidono con scenari AI-Based già classificati come dipendenti da un servizio esterno; nessun nuovo incidente post-modifica è stato registrato.

6 Valutazione Complessiva

La valutazione finale combina le metriche di coverage, gli avvisi emersi durante l’esecuzione e i criteri di accettazione stabiliti nel Master Test Plan.

6.1 Branch Coverage

La Tabella 6.1 confronta i valori misurati dopo le modifiche con quelli fissati nel Master Test Plan.

Tabella 6.1. Confronto della branch coverage

Suite	Pre-modifica	Post-modifica	Variazione
Unit test Python	85%	87%	+2 p.p.
Integration test Python	34%	51%	+17 p.p.

Entrambi i livelli superano la baseline. La crescita della coverage di integrazione non viene utilizzata come sostituto dei test funzionali, ma come evidenza complementare alla regressione per componente e livello.

6.2 Osservazioni Non Bloccanti

Durante le esecuzioni sono comparsi avvisi relativi a dipendenze o configurazioni deprecate. Nessuno di essi ha prodotto una divergenza funzionale o impedito il completamento delle suite.

Tabella 6.2. Avvisi osservati durante il testing

Origine	Osservazione	Valutazione
HTTPIX	Deprecazione dello shortcut app nei test di integrazione.	Debito tecnico dei test.
Node.js	Avviso di deprecazione del modulo punycode.	Dipendenza transitiva; test positivi.
Docker Compose	Chiave version dichiarata obsoleta.	Configurazione valida e build riuscito.
Pydantic	Ridenominazione di schema_extra in json_schema_extra.	Contratto e risposta HTTP preservati.
Cypress	Risoluzione tsconfig-paths non attivata per assenza di baseUrl.	Compilazione e scenari positivi.

Gli avvisi sono candidati a manutenzione futura, ma non costituiscono incidenti di test né violazioni dei criteri di accettazione correnti.

6.3 Applicazione dei Criteri di Accettazione

Tabella 6.3. Valutazione rispetto al Master Test Plan

Criterio	Evidenza	Esito
Preservazione della coverage	Unit 87% contro 85%; integration 51% contro 34%.	Soddisfatto
Assenza di bug critici	Tutte le suite e gli smoke test in scope terminano senza fallimenti; nessun incidente risolto è stato riaperto.	Soddisfatto
Allineamento ai contratti esistenti	Gateway, servizio statico e servizio di report rispondono correttamente in locale e Docker.	Soddisfatto
Assenza di regressioni funzionali	Corpus CLI invariato, 32 test frontend e 22 scenari E2E in scope positivi.	Soddisfatto

Poiché tutti i criteri sono soddisfatti, CR1, CR2, CR3 e CR4 risultano accettate. La decisione comprende i deliverable documentali previsti da CR2 e CR3, le verifiche sui DIS di CR3 e CR4 e la regressione dei componenti presenti nei FPIS.

6.4 Conclusioni

Il sistema post-modifica soddisfa i requisiti funzionali e adattivi delle quattro Change Request. Il Call Graph è integrato nel core e configurabile dalla CLI; i servizi in scope sono avviabili senza modifiche sia localmente sia in Docker; la Web Application presenta il grafo con le interazioni, i filtri e le esportazioni richieste.

La baseline funzionale è preservata, la branch coverage non diminuisce e non emergono bug critici o regressioni nei contratti osservabili. Gli scenari AI-Based restano non applicabili per scelta di perimetro e gli avvisi di deprecazione sono non bloccanti. Le evidenze raccolte consentono di concludere con esito positivo l’implementazione e la validazione delle Change Request.